

Rational Matrix Policy Optimization for Rational Feed-Forward Language Models

Anonymous authors
Paper under double-blind review

Abstract

1 Introduction

2 Related Work

3 Method

We consider the problem of training a Transformer language model in which each feed-forward block is replaced by a rational nonlinear module. Let $\mathcal{L}(\theta)$ denote the training objective. A standard optimizer applies a rule of the form $\theta^{t+1} = \mathcal{U}(\theta^t, \nabla \mathcal{L}(\theta^t))$ to all parameters according to tensor shape or parameter group. Rational Matrix Policy Optimization instead uses the algebraic structure of the rational feed-forward block to define separate update rules for its input matrix, nonlinear coefficients, output matrix, and gauge degree of freedom. The base learning-rate schedule is not changed; all additional behavior is local to the rational block.

3.1 Rational local-basis feed-forward block

Let $x_l \in \mathbb{R}^d$ be the residual-stream input to layer l . The RLB feed-forward block first projects x_l into G hidden groups of width m :

$$z_l = A_l x_l, \quad z_{l,g} = \text{group}_g(z_l), \quad g = 1, \dots, G, \quad (1)$$

where $A_l \in \mathbb{R}^{Gm \times d}$ is the input matrix. Each group is written in polar form using an RMS radius,

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon}, \quad u_{l,g} = z_{l,g} / r_{l,g}. \quad (2)$$

The group nonlinearity is a learned rational function $R_{l,g}$ evaluated at the normalized coordinate and rescaled by the radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad y_l = B_l \text{concat}_{g=1}^G h_{l,g}, \quad (3)$$

with output matrix $B_l \in \mathbb{R}^{d \times Gm}$. The rational map may include a base rational curve and local basis coefficients, but the block remains a single-branch feed-forward module: it contains no multiplicative gate, no value/gate split, and no hidden SiLU path. The parameterization therefore exposes the ordered roles

$$A_l : \text{input-domain selection}, \quad (4)$$

$$R_{l,g} : \text{rational shape}, \quad (5)$$

$$B_l : \text{feature recombination}. \quad (6)$$

3.2 Gauge symmetry and optimizer desiderata

The RMS normalization creates an exact positive gauge symmetry. For any $a_g > 0$, let $D_l(a)$ be block diagonal with g th block $a_g I_m$. Then

$$A'_l = D_l(a) A_l, \quad B'_l = B_l D_l(a)^{-1} \quad (7)$$

represents the same feed-forward function as (A_l, B_l) . This follows because $z'_{l,g} = a_g z_{l,g}$ and $r'_{l,g} = a_g r_{l,g}$, while $u'_{l,g} = u_{l,g}$; hence $h'_{l,g} = a_g h_{l,g}$ and the inverse scale in B'_l cancels the change.

This invariance implies that Euclidean parameter distance is not a faithful proxy for functional change. An optimizer can spend update energy changing the gauge representative without changing the model function. MatrixPolicy is designed around three desiderata:

1. update A_l , $R_{l,g}$, and B_l according to their distinct roles;
2. use live group statistics to allocate matrix movement across rational groups;
3. rebalance the positive gauge after the optimizer step without changing the represented function.

3.3 Parameter partition

The trainable parameters are partitioned as

$$\theta = \theta_{\text{base}} \cup \theta_{\text{coeff}} \cup \{M_{l,\text{in}}, M_{l,\text{out}}\}_{l=1}^L, \quad M_{l,\text{in}} = A_l, \quad M_{l,\text{out}} = B_l. \quad (8)$$

Here θ_{base} contains embeddings, attention, normalization, and other non-RLB parameters. These parameters are updated by a base optimizer $\mathcal{U}_{\text{base}}$. The rational coefficients θ_{coeff} are updated by a coefficient rule $\mathcal{U}_{\text{coeff}}$. The matrices A_l and B_l are updated by the MatrixPolicy rule defined below. This partition makes the optimizer a local extension of a conventional training procedure rather than a replacement for the whole-model optimizer.

3.4 Role-depth matrix policy

Let $d_l = l/(L-1)$ denote normalized layer depth. MatrixPolicy assigns separate role-depth factors to the input and output matrices:

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(d_l - 0.5), 0.55, 1.40), \quad (9)$$

$$\rho_{\text{out}}(l) = \text{clip}(1 + 1.00(d_l - 0.5), 0.55, 1.40). \quad (10)$$

The corresponding adaptive-step multiplier is

$$a_{\text{role}}(l, r) = \max(0.10, 1 + \alpha_{\text{role}}(\rho_r(l) - 1)), \quad \alpha_{\text{role}} = 1.20, \quad (11)$$

where $r \in \{\text{in}, \text{out}\}$. This factor encodes the prior that input matrices primarily choose rational domains, while output matrices primarily choose how rational features are recombined. The depth dependence allows this role preference to vary across the Transformer stack.

3.5 Matrix update rule

For each RLB matrix $M_{l,r}$, the update is a mixture of an adaptive first-order step and an early matrix-normalized step. Let η_t be the shared base learning rate and let $\mu_{\text{base}}(t)$ be a smooth window that is nonzero only in the early phase of training. The mixture coefficient is

$$\mu(l, r, t) = \text{clip}(\mu_{\text{base}}(t)\rho_r(l)s_{\text{stat}}(l, t), 0, 0.75). \quad (12)$$

The matrix update is

$$M_{l,r}^{t+1} = M_{l,r}^t + \Delta_{\text{adapt}}(M_{l,r}; \eta_t a_{\text{mat}}(l, r, t)[1 - \mu(l, r, t)]) + \Delta_{\text{mat}}(M_{l,r}; \eta_t a_{\text{matnorm}}\mu(l, r, t)), \quad (13)$$

where Δ_{adapt} is the Adam-style adaptive component (Kingma & Ba, 2015) and Δ_{mat} is the matrix-normalized component. The local scale is

$$a_{\text{mat}}(l, r, t) = \text{clip}(3.0 a_{\text{role}}(l, r) a_{\text{stat}}(l, r, t), 0.40, 4.0), \quad a_{\text{matnorm}} = 1.0. \quad (14)$$

The matrix-normalized component is temporary: it is used to move matrix subspaces efficiently early in training, then decays so that late optimization is dominated by the adaptive component and the gauge-controlled parameterization.

3.6 On-policy group statistics

MatrixPolicy uses statistics measured along the optimization trajectory. For each rational group, define relative gradient pressures

$$p_{\text{in},g} = \frac{\text{rms}(\nabla A_{l,g})}{\text{rms}(A_{l,g})}, \quad p_{\text{out},g} = \frac{\text{rms}(\nabla B_{l,g})}{\text{rms}(B_{l,g})}, \quad p_{\text{rat},g} = \text{rms}(\nabla \theta_{\text{coeff},g}). \quad (15)$$

Let $q_{\text{in},g}$, $q_{\text{out},g}$, and $q_{\text{rat},g}$ be exponential moving averages of these quantities. The policy uses

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}, \quad (16)$$

$$\alpha_g = \log q_{\text{rat},g} - \frac{1}{2}(\log q_{\text{in},g} + \log q_{\text{out},g}). \quad (17)$$

The pressure imbalance π_g measures whether the input or output side of a rational group is moving faster. The activity score α_g measures rational coefficient movement relative to matrix movement.

3.7 Group-stat preconditioning

The group-stat policy defines a centered multiplicative preconditioner for each RLB matrix group:

$$c_g = c_{\text{gain},g} c_{\text{pressure},g} c_{\text{activity},g}, \quad c_g \leftarrow c_g / \text{geomean}(c), \quad c_g \leftarrow \text{clip}(c_g, 0.75, 1.35). \quad (18)$$

The gain term equalizes derivative or output activity,

$$c_{\text{gain},g} = \left(\frac{\text{geomean}(k)}{k_g} \right)^{0.20}, \quad (19)$$

where k_g is derivative RMS for the input matrix and output RMS for the output matrix. The pressure term is role-dependent:

$$c_{\text{pressure},g} = \exp(0.10 d_{\text{pressure},g}), \quad d_{\text{pressure},g} = \begin{cases} -\pi_g, & r = \text{in}, \\ +\pi_g, & r = \text{out}. \end{cases} \quad (20)$$

The activity term damps groups whose rational coefficients move too aggressively relative to the adjacent matrices:

$$c_{\text{activity},g} = \exp[-0.20 \text{ReLU}((\alpha_g - 0.05)/0.45)]. \quad (21)$$

The scale c_g is applied row-wise to A_l gradients and column-wise to B_l gradients. Centering by the geometric mean keeps the group-stat rule from acting as a hidden global learning-rate change.

3.8 Gauge rebalance

After the parameter update, MatrixPolicy applies a function-preserving gauge rebalance. Define the matrix-norm log-ratio

$$\gamma_g = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (22)$$

The target log-ratio combines rational derivative/output statistics and live input-output pressure:

$$\tau_g = \frac{1}{2}(\log d_g^{\text{gain}} - \log o_g^{\text{gain}}) + \beta_{\text{pressure}}(\log q_{\text{in},g} - \log q_{\text{out},g}). \quad (23)$$

The correction is clipped,

$$\ell_g = \frac{1}{2}(\tau_g - \gamma_g), \quad \ell_g \leftarrow \text{clip}(s(t, l) a_g^{\text{activity}} \ell_g, -\ell_{\text{max}}, \ell_{\text{max}}), \quad (24)$$

and the exact gauge transform is applied:

$$A_{l,g} \leftarrow \exp(\ell_g) A_{l,g}, \quad B_{l,g} \leftarrow B_{l,g} / \exp(\ell_g). \quad (25)$$

This operation changes the representative of the gauge class but not the network function. It is therefore a conditioning step, not an additional descent step.

3.9 Algorithm

Algorithm 1: Rational Matrix Policy step. Given parameters θ^t , gradients $\nabla\mathcal{L}(\theta^t)$, and base learning rate η_t :

1. Update group-pressure EMAs q_{in} , q_{out} , and q_{rat} .
2. Compute role-depth factors $\rho_{\text{in}}(l)$ and $\rho_{\text{out}}(l)$.
3. Compute centered group-stat scales c_g and precondition RLB matrix gradients.
4. Update θ_{base} with $\mathcal{U}_{\text{base}}$ and update θ_{coeff} with $\mathcal{U}_{\text{coeff}}$.
5. Update each A_l and B_l using the role-depth matrix mixture in Section 3.5.
6. Apply the exact gauge rebalance $A_{l,g} \leftarrow e^{\ell_g} A_{l,g}$, $B_{l,g} \leftarrow B_{l,g}/e^{\ell_g}$.

The algorithm is fully specified by the RLB decomposition and the shared base optimizer. Its nonstandard operations are not global schedule changes: each one corresponds to a structural feature of the rational block.

4 Experiments

5 Discussion

6 Limitations

7 Conclusion

References

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In International Conference on Learning Representations, 2015.